Recitation: Quiz 2 Review

Concepts and Papers from CNNs to Scaling Laws

Timothy Obiso

Department of Computer Science
Brandeis University

COSI 115b — Spring 2026

Today's Plan

1. CNNs & RNNs
2. Attention
3. Transformers
4. Large Language Models
5. Post-training & RL
6. Scaling Laws
7. Mixture-of-Experts

For each: the core idea, the key paper(s), and what they contributed.

CNNs for Text — Kim (2014)

Before Transformers, convolutional networks were a strong, simple baseline for sentence-level tasks.

Key paper: Kim, "Convolutional Neural Networks for Sentence Classification" (EMNLP 2014).

Contribution:

- A simple CNN on top of pretrained word embeddings gets competitive results on many classification benchmarks
- Popularized multi-width filters + max-over-time pooling for text

How a Text CNN Works

1. Embed each word $\Rightarrow$ matrix of shape $n \times d$
2. Apply 1D convolutions of widths $k \in \{3, 4, 5\}$ in parallel
   - Each filter of width k captures k-gram patterns
   - Many filters per width
3. **Max-over-time pool** each filter's output $\Rightarrow$ one scalar per filter
4. Concatenate $\Rightarrow$ fixed-size vector, feed to a classifier

Convolution output length: kernel size k, stride 1, no padding, input length $n \Rightarrow$ output length $n - k + 1$.

Why Max-Over-Time?

Max-pooling collapses the filter's output to one scalar regardless of sentence length. This is what makes CNNs work for variable-length input.

Recurrent Neural Networks

At each timestep:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

A recurrent network processes a sequence one token at a time, passing a hidden state forward.

Why RNNs instead of CNNs?

- Naturally handle variable-length input
- Model dependencies across arbitrary distances (in principle)

But: vanilla RNNs suffer from the **vanishing gradient** problem.

Vanishing & Exploding Gradients

Backpropagation through time multiplies gradients by W_h repeatedly:

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{i=1}^{k} \frac{\partial h_{t-i+1}}{\partial h_{t-i}}$$

- $\|W_h\| < 1$: gradients shrink exponentially $\Rightarrow$ **vanishing**
- $\|W_h\| > 1$: gradients grow exponentially $\Rightarrow$ **exploding**

Consequence

Vanilla RNNs can't learn long-range dependencies. Exploding gradients are usually fixable with clipping; vanishing gradients require a structural fix — which is what LSTMs and GRUs provide.

LSTMs

Long Short-Term Memory (Hochreiter & Schmidhuber, 1997) adds a separate cell state c_t and three sigmoid gates:

- **Forget gate** f_t: how much of old cell state to keep
- **Input gate** i_t: how much new candidate info to write
- **Output gate** o_t: how much cell state to expose as hidden

Cell state update:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

Why It Works

The update is *additive*, not multiplicative. Gradients flow through c_t along a "constant error carousel" — the path that mitigates vanishing gradients.

GRUs — Cho et al. (2014)

Key paper: Cho, van Merriënboer, Bahdanau, Bengio, "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation."

Contribution: introduces

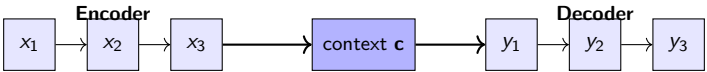
- The **GRU** (gated recurrent unit) — simpler than LSTM
- The **encoder–decoder** framework for neural MT

GRU gates:

- **Reset gate** r_t: how much of past hidden state to ignore
- **Update gate** z_t: interpolates between old and new hidden state

Fewer parameters than LSTM, no separate cell state, often comparable performance.

Encoder–Decoder for Machine Translation



- **Encoder** reads source sentence, compresses it into a fixed-dim context vector (final hidden state)
- **Decoder** conditions on the context and generates the target autoregressively

The Bottleneck

A single fixed-dim vector has to carry an entire sentence's worth of information. This motivates **attention**.

The Attention Idea

"Don't squeeze the whole source into one vector. Let the decoder **look back** *at every source position."*

Instead of a single context vector, attention gives the decoder a **soft, differentiable lookup** over all encoder positions.

Key paper: Vaswani et al., "Attention Is All You Need" (NeurIPS 2017).

Contribution: showed that you can replace recurrence entirely with attention and get better results, faster training (parallelism), and longer-range modeling.

Queries, Keys, and Values

Attention is a differentiable lookup with three roles:

- **Query** Q — "what am I looking for?"
- **Key** K — "what do I have?"
- **Value** V — "what do I return if you match me?"

Each input token produces a query, a key, and a value (via learned linear projections).

Attention scores between positions are computed from queries and keys; the values are what get aggregated.

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Steps:

1. Compute similarity: $QK^\top$ (dot product between each query and each key)
2. Scale by $\sqrt{d_k}$
3. Softmax over keys $\Rightarrow$ attention weights that sum to 1
4. Weighted sum of values

Why Scale by $\sqrt{d_k}$?

Without scaling, large d_k produces large-magnitude dot products, pushing softmax into saturation where gradients vanish. Scaling keeps softmax in a useful regime regardless of head dimension.

Multi-Head Attention

Instead of one attention with $d_k = d_{\text{model}}$, run h attentions in parallel with $d_k = d_{\text{model}}/h$, concatenate outputs.

Why?

- Different heads attend to different **linguistic relationships** — e.g., syntactic dependency, coreference, positional proximity
- Same total compute as one big head, richer representation
- A single head averages everything into one attention distribution and loses the diversity

For $d_{\text{model}} = 512$ and $h = 8$ heads: $d_k = 64$ per head.

Self-Attention vs. Cross-Attention

Self-attention	Q, K, V all from the *same* sequence. A position attends to other positions in its own sequence.
Cross-attention	Q from one sequence (e.g., decoder), K, V from another (e.g., encoder). Used in encoder–decoder architectures.

Masking variants of self-attention:

- **Bidirectional**: each position sees all positions (BERT)
- **Causal / masked**: position i sees only positions $\leq i$ (GPT)

The Transformer — Vaswani et al. (2017)

The original Transformer is an **encoder–decoder** architecture for machine translation.
Each block is just two sublayers:

1. Multi-head self-attention (+ residual + LayerNorm)
2. Feed-forward network (+ residual + LayerNorm)

Why it worked so well:

- Fully parallel over positions (unlike RNNs) $\Rightarrow$ much faster training
- Constant path length between any two positions $\Rightarrow$ long-range dependencies
- Scales gracefully with more layers, heads, data, compute

Positional Encodings

Self-attention is permutation-equivariant. It doesn't know token order.

Solution: add position information to the token embedding before feeding to the network.
Variants:

- **Sinusoidal** (Vaswani et al., 2017): fixed, deterministic function of position

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

- **Learned** (BERT, GPT-2): position embedding table, trained jointly
- **Rotary / ALiBi** (modern): inject position *into* the attention computation

The original Transformer used sinusoidal; BERT switched to learned.

Residual Connections and LayerNorm

Residual connections:

$$\text{output} = x + \text{sublayer}(x)$$

- Direct gradient path through many layers
- Sublayer can learn a near-identity mapping if not useful $\Rightarrow$ adding layers can't hurt
- Makes very deep stacks trainable

Layer normalization:

- Normalizes activations per-position across feature dim
- Stabilizes training dynamics
- Essentially every modern Transformer uses some variant (pre-norm, post-norm, RMSNorm, etc)

BERT — Devlin et al. (2019)

Key paper: "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" (NAACL 2019).
Architecture: encoder-only, bidirectional self-attention.
Pretraining objective: masked language modeling (MLM)

- Mask ~15% of input tokens
- Model predicts them using full bidirectional context
- Loss computed only on masked positions

What It Unlocked

Strong general-purpose contextual representations. Fine-tune a small head on top for classification, NER, QA, NLI — and get state-of-the-art across the board.

GPT — Radford et al. (2018)

Key paper: "Improving Language Understanding by Generative Pre-Training."
Architecture: decoder-only, causal (triangular) self-attention.
Pretraining objective: causal language modeling (CLM)

- Predict next token given past
- Loss computed on every position
- Autoregressive factorization: $P(x_1, \ldots, x_n) = \prod_t P(x_t \mid x_{<t})$

Later scaled up: GPT-2 (1.5B), GPT-3 (175B), etc. The line of work that established decoder-only as the dominant architecture for LLMs.

Three Transformer Families

- **Encoder-only** (BERT, RoBERTa, DeBERTa)
  - Bidirectional attention
  - Best for classification, tagging, retrieval, embeddings
  - Cannot generate autoregressively
- **Decoder-only** (GPT, Llama, Mistral)
  - Causal attention
  - Best for open-ended generation, dialogue, prompting
  - Can also do classification via prompting
- **Encoder–decoder** (original Transformer, T5, BART)
  - Bidirectional encoder, causal decoder, cross-attention
  - Best for sequence-to-sequence (translation, summarization)

Scaling Up Language Models

From 2018 onward: bigger models, more data, more compute. Surprising capabilities emerged along the way.

Key question: what does a large language model trained only on next-token prediction actually know how to do?

Two papers on the reading list answer different pieces of that:

- Brown et al. (2020) — GPT-3 and in-context learning
- Wei et al. (2022) — chain-of-thought prompting

In-Context Learning — Brown et al. (2020)

Key paper: "Language Models are Few-Shot Learners" (NeurIPS 2020).
The surprise: you can get GPT-3 to do a new task just by *showing it examples in the prompt*. No weight updates. No fine-tuning.
Three evaluation regimes:

- **Zero-shot**: task description only, no examples
- **One-shot**: task description + one example, then target
- **Few-shot**: task description + several examples, then target

Key Point

In-context learning is **inference-only**. Weights are frozen. Learning happens purely through conditioning on the prompt.

In-Context Learning Grows With Scale

Central finding: the gap between few-shot and zero-shot performance **grows with model size**.

- Small models: few-shot barely helps
- Large models (175B): few-shot gives substantial gains on many tasks

Interpretation

In-context learning is itself an emergent capability of scale — not a property of any language model.

What this changed: the focus of the field shifted from task-specific fine-tuning to prompt engineering and general-purpose models.

Chain-of-Thought — Wei et al. (2022)

Key paper: "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models" (NeurIPS 2022).
Idea: prompt the model to produce **intermediate reasoning steps** before the final answer.
Two common forms:

- **Few-shot CoT**: examples in the prompt include step-by-step work
- **Zero-shot CoT**: append "Let's think step by step."

Large gains on multi-step reasoning: arithmetic word problems, commonsense, logic.

CoT is an Emergent Ability

Below a certain model scale, CoT prompting doesn't help — and can hurt.
Why?

- Small models produce incoherent reasoning chains
- Noise in the scratchpad distracts from the final answer

At large scale ($\sim$60B+ in the paper):

- Chains become coherent
- Model effectively has more serial compute to solve multi-step problems

Pattern to Recognize

This is part of a broader pattern of **emergent abilities**: capabilities that appear suddenly at scale and can't be extrapolated from smaller models.

Why Post-Training?

A pretrained LM can predict the next token — but that's not the same as being **useful**.

Base model problems:

- Continues prompts rather than answering them
- No preference for being helpful, harmless, honest
- Will happily hallucinate, refuse wrong things, or repeat itself

Post-training Goal

Align the pretrained model with human preferences, without retraining from scratch.

InstructGPT — Ouyang et al. (2022)

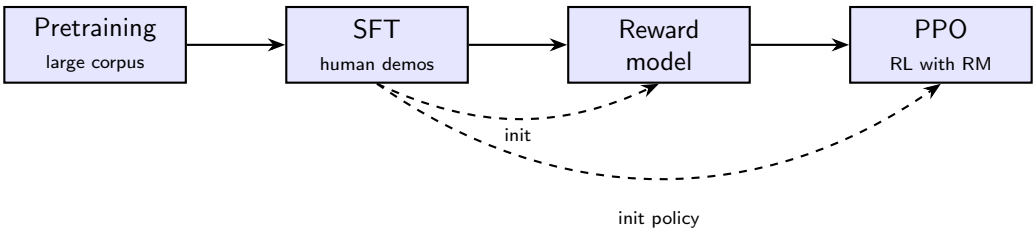
Key paper: "Training Language Models to Follow Instructions with Human Feedback."
Three-stage training pipeline:

1. **Supervised fine-tuning (SFT)**: fine-tune the pretrained LM on human-written demonstrations of desired behavior
2. **Reward modeling**: train a separate model to score outputs, from pairwise human preference judgments
3. **RL with PPO**: optimize the LM against the reward model

Headline Result

The 1.3B-parameter InstructGPT was preferred by human evaluators over the 175B-parameter base GPT-3. Alignment can matter more than raw scale.

The RLHF Pipeline



Detail Worth Knowing

The reward model and the RL-tuned policy are both *initialized* from the SFT model, not trained from scratch.

Why a Separate Reward Model?

Why not ask humans to rate outputs directly during RL?

- **Cost**: RL requires scoring millions of generations. Human labeling at that scale is impossible.
- **Consistency**: humans are better at *comparing* two outputs than at rating them on a scale. Pairwise preferences are less noisy.
- **Generalization**: the RM must score novel outputs generated during training. A learned model generalizes from the fixed preference dataset.
- **Speed**: RL needs a reward signal at every gradient step. Humans can't keep up.

PPO — Schulman et al. (2017)

Key paper: "Proximal Policy Optimization Algorithms."
A policy-gradient method that's both **stable** and **sample-efficient**.
The trick — clipping:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}\Big[\min\big(r_t\hat{A}_t,\ \text{clip}(r_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big]$$

where $r_t(\theta) = \pi_\theta(a|s)/\pi_{\text{old}}(a|s)$ is the policy ratio.

What Clipping Buys Us

The policy can't move too far in a single update. Trust-region-like stability without the complexity of TRPO.

RLHF also adds a **KL penalty** against the SFT model to prevent drift and reward hacking.

Kaplan et al. (2020)

Key paper: "Scaling Laws for Neural Language Models."
Finding: test loss follows a **power-law** in three quantities:

$$L(N) \propto N^{-\alpha}, \quad L(D) \propto D^{-\beta}, \quad L(C) \propto C^{-\gamma}$$

where N = parameters, D = tokens, C = compute.
What "power-law" actually means:

- Each doubling of N gives a *constant relative* (multiplicative) reduction in loss
- *Not* a constant absolute reduction — absolute gains shrink as loss shrinks
- Loss vs. N on log-log axes $\Rightarrow$ a straight line

Scaling Laws in Practice

Why do labs run scaling-law experiments?

- Run many small models cheaply
- Fit a power-law
- **Extrapolate** to predict the loss of a single very large run
- Pick the best (N, D) pair for your compute budget

Compute budget approximation: $C \approx 6ND$.
Given a fixed C, this tells you how to trade off model size and tokens.

The Takeaway

Scaling laws aren't just theory — they're the primary planning tool at frontier labs.

Chinchilla — Hoffmann et al. (2022)

Key paper: "Training Compute-Optimal Large Language Models."
Finding: the Kaplan recipe was wrong about allocation.

- Kaplan (2020): scale N fast, let D lag
- Chinchilla (2022): N and D should grow *together* — roughly $D \approx 20N$

The Experiment

Chinchilla (70B parameters, 1.4T tokens) vs. Gopher (280B parameters, 300B tokens) — same compute budget.
Chinchilla outperformed Gopher across the board. Previous models were **undertrained**.

Modern Practice

Today's LLMs are typically trained on *far* more tokens than Chinchilla-optimal.
Example: Llama 3 (8B parameters) trained on roughly 15 trillion tokens — well past the $D \approx 20N$ ratio.

Why?

- Inference cost scales with N, not D
- A smaller, "over-trained" model is cheaper per query
- For models that will serve many queries, the trade is worth it: you spend more training compute once, save serving compute forever

Chinchilla-optimality is about **training** efficiency. Real deployment often trades that for **inference** efficiency.

Why Mixture-of-Experts?

Scaling laws say: more parameters $\Rightarrow$ lower loss. But more parameters also means more compute *per token*.

Mixture-of-Experts (MoE) breaks that link:

- Replace a single large feed-forward layer with E smaller "expert" FFNs
- For each token, a **router** picks only k experts (typically $k = 1$ or 2)
- Only the chosen experts' weights are used for that token

The Payoff

Total parameters grow with E (more capacity, lower loss). Compute per token grows only with k (typically constant). You get a huge model at a small model's inference cost.

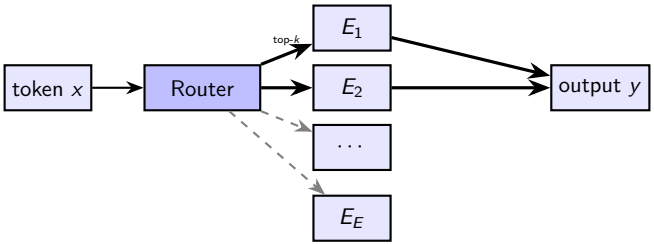
The Foundational Paper — Shazeer et al. (2017)

Key paper: Shazeer, Mirhoseini, Maziarz, Davis, Le, Hinton, Dean, "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer" (ICLR 2017).
Contribution:

- Sparsely-gated MoE layer: top-k routing with a learned gating network
- Showed MoE scales to **hundreds of billions of parameters** with per-token compute barely increased
- Introduced the load-balancing loss used by nearly every MoE since

Pre-dates Transformers — applied to LSTM language models originally — but the recipe carried over directly.

Anatomy of an MoE Layer



Per token:

1. Router computes a score for each of the E experts
2. Pick the top-k experts (typically $k = 1$ or 2)
3. Run the token through just those experts
4. Output is a gated weighted sum of their outputs

The Router

The router is typically a single linear layer plus softmax:

$$g(x) = \mathrm{softmax}(W_r x)$$

Top-k routing keeps the largest k entries and zeroes the rest:

$$y = \sum_{i \in \text{top-}k(g(x))} g_i(x) \cdot E_i(x)$$

Design choices that matter:

- $k = 1$ (Switch Transformer): cheapest, hardest to train stably
- $k = 2$ (original Shazeer, Mixtral): more robust, slightly more compute
- **Noise during training**: adds exploration so the router doesn't collapse onto a single expert

The Load-Balancing Problem

Without constraints, the router tends to **collapse**: it sends nearly all tokens to a small subset of experts.

Why that's bad:

- Most experts get no gradient signal $\Rightarrow$ they're dead weight
- The model's effective capacity is much smaller than E experts
- Parallelism across devices (each expert on a different GPU) becomes useless — one GPU does all the work

The Fix: Load-Balancing Loss

Add an auxiliary loss that penalizes uneven expert usage over a batch. Encourages each expert to see roughly $1/E$ of the tokens. This is **not** a constraint at inference — just during training.

Switch Transformer — Fedus et al. (2022)

Key paper: Fedus, Zoph, Shazeer, "Switch Transformer: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity" (JMLR 2022).

Contribution:

- Showed $k = 1$ routing ("hard switch") works well and is cheaper than $k = 2$
- Scaled MoE Transformers to **1.6 trillion parameters**
- Careful engineering: capacity factors, routing precision in fp32, expert parallelism

Why It Mattered

Demonstrated that MoE isn't just a clever trick — it's a practical recipe for reaching parameter counts that are infeasible with dense models, while keeping inference cost manageable.

Modern MoE: Mixtral and DeepSeek

Post-2023, MoE became standard in frontier open models.

Mixtral 8x7B (Mistral AI, 2024):

- 8 experts per layer, $k = 2$
- Total parameters: $\sim$47B. Active per token: $\sim$13B.
- Matches or beats Llama 2 70B on most benchmarks at a fraction of the inference cost

DeepSeek-V3 / DeepSeek MoE (2024–2025):

- Hundreds of experts, fine-grained routing
- Shared experts (always active) + routed experts (top-k)
- 671B total, $\sim$37B active per token

MoE vs. Dense: The Trade-off

	Dense	**MoE**
Params per token	all of them	only top-k experts
Total params	= per-token params	$\gg$ per-token params
Training FLOPs	proportional to total params	proportional to active params
Memory (inference)	proportional to total params	proportional to total params
Parallelism style	tensor / pipeline	expert parallelism

Key Point

MoE reduces *compute*, not *memory*. You still have to fit all the experts somewhere — it doesn't give you a free lunch on GPU RAM.

Open Problems in MoE

MoE is powerful but far from solved.

- **Router stability**: top-k is discrete $\Rightarrow$ routing decisions have no gradient. Training is sensitive to initialization, capacity factors, and load-balance weight.
- **Fine-tuning is fragile**: MoEs can forget or collapse routing patterns during fine-tuning and RLHF.
- **Expert specialization is mysterious**: despite lots of probing, experts don't consistently specialize by topic, language, or syntax.
- **Inference serving is tricky**: batching across users with different routing patterns is harder than for dense models.

Active area of research — including work at Brandeis on alternative routing mechanisms.

Papers Covered

- Kim (2014) — CNN for sentence classification
- Cho et al. (2014) — GRU, encoder–decoder
- Vaswani et al. (2017) — Transformer, attention
- Devlin et al. (2019) — BERT, MLM
- Radford et al. (2018) — GPT, CLM
- Brown et al. (2020) — GPT-3, in-context learning
- Wei et al. (2022) — chain-of-thought prompting
- Ouyang et al. (2022) — InstructGPT, RLHF
- Schulman et al. (2017) — PPO
- Kaplan et al. (2020) — scaling laws
- Hoffmann et al. (2022) — Chinchilla, compute-optimal training
- Shazeer et al. (2017) — sparsely-gated MoE
- Fedus et al. (2022) — Switch Transformer

Questions?

Good luck on Quiz 2.